



P2P Systems and Blockchains

-Lab-

Damiano Di Francesco Maesa
9/3/2026
damiano.difrancesco@unipi.it



UNIVERSITÀ
DI PISA

Lecture 2 - Bitcoin lab

Bitcoin Tools

Bitcoin Core

<https://bitcoin.org/en/download>

bitcoinj

<https://bitcoinj.org/>

jar download

<https://search.maven.org/remotecontent?filepath=org/bitcoinj/bitcoinj-core/0.16/bitcoinj-core-0.16.jar>

Javadoc

<https://bitcoinj.org/javadoc/0.17/>



bitcoinj - connection example

Ask for a testnet example block

<https://blockstream.info/testnet/block/00000000000000adc6423b570d751efcdf5e019d3d955fee155c28925913cb667>

bitcoinj - connection example

```

1 public static void connectionTest() throws InterruptedException {
2     NetworkParameters netParams = TestNet3Params.get();
3
4     BlockStore blockStore = new MemoryBlockStore(netParams.getGenesisBlock());
5     Blockchain blockChain;
6     try {
7         blockChain = new Blockchain(netParams.network(), blockStore);
8         PeerGroup peerGroup = new PeerGroup(netParams, blockChain);
9         peerGroup.setUserAgent("Sample App", "1.0");
10        peerGroup.addPeerDiscovery(new DnsDiscovery(netParams));
11        peerGroup.start();
12        Thread.sleep(10000);
13        printNetStats(peerGroup);
14        for (Peer p: peerGroup.getConnectedPeers()) {
15            System.out.println(p.getAddr());
16        }
17        while (peerGroup.getConnectedPeers().isEmpty())
18            Thread.sleep(5000);
19        Sha256Hash blockHash = Sha256Hash.wrap("000000000000adc6423b570d751efcdf
20        5e019d3d955fee155c28925913cb667");
21        Block block;
22        boolean flag = true;
23        try {
24            while (flag) {
25                Peer pFirst = peerGroup.getConnectedPeers().get(peerGroup
26                .getConnectedPeers().size() - 1);
27                Future < Block > future = pFirst.getBlock(blockHash);
28                block = future.get(5, TimeUnit.SECONDS);
29                System.out.println("Here is the block: " + block);
30                flag = false;
31            }
32        } catch (TimeoutException ex) {
33            //do nothing, just try a new peer
34        } catch (ExecutionException ex) {
35            //do nothing, just try a new peer
36        }
37    }
38 }

```

```

35         Thread.sleep(10000);
36         printNetStats(peerGroup);
37         peerGroup.stop();
38     } catch (BlockStoreException ex) {
39         System.getLogger(P2PBClab1project.class.getName()).log(System.Logger
.Level.ERROR, (String) null, ex);
40     }
41 }
42
43 public static void printNetStats(PeerGroup peerGroup) {
44     System.out.println("\n\nNETWORK INFO:");
45     System.out.println("Max conenctions: " + peerGroup.getMaxConnections());
46     System.out.println("Current conenctions: " + peerGroup.numConnectedPeers());
47     System.out.println("Chain height: " + peerGroup.getMostCommonChainHeight());
48     System.out.println("\n\n");
49 }

```

bitcoinj - addresses

Mainnet

Legacy: P2PKH start with "1" , P2SH start with "3"

Nested SegWit: P2SH-P2WPKH and P2SH-P2WSH start with "3" (transitional only)

SegWit: P2WPKH and P2WSH start with "bc1q"

Taproot: P2TR start with "bc1p"

bitcoinj - addresses

```
1 public static void createNewAddr() {  
2  
3     NetworkParameters netParams1 = TestNet3Params.get();  
4  
5     ECKey key = new ECKey();  
6     System.out.println("We created key " + key);  
7     Address addressFromKey = key.toAddress(ScriptType.P2PKH,  
8     netParams1.network());  
9  
10    System.out.println("On the " + netParams1.network() + " network  
11    , we can use this address " + addressFromKey);  
12  
13    NetworkParameters netParams2 = MainNetParams.get();  
14    addressFromKey = key.toAddress(ScriptType.P2PKH, netParams2  
15    .network());  
16    System.out.println("On the " + netParams2.network() + " network  
17    , we can use this address " + addressFromKey);  
18 }
```

bitcoinj - addresses

EXERCISE 1:

create a vanity address starting with a chosen string.

bitcoinj - blocks

Let's inspect the Bitcoin genesis block

0x00000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f

https://en.bitcoin.it/wiki/Genesis_block

bitcoinj - blocks

```
1 //Get Bitcoin Genesis
2 //0x00000000000019d6689c085ae165831e934ff763ae46a2a6c172b3f1b60a8ce26f
3 public static void getBtcGenesis() throws InterruptedException {
4     NetworkParameters netParams = MainNetParams.get();
5
6     Block genesis = netParams.getGenesisBlock();
7
8     System.out.println(genesis);
9
10    TransactionInput txIn = genesis.getTransactions().get(0).getInput(0);
11
12    System.out.println(bytesToHex(txIn.getScriptBytes()));
13
14    String message = hexToAscii(bytesToHex(txIn.getScriptBytes()));
15
16    System.out.println(message);
17
18    printBlockInfo(genesis);
19
20    printTxInfo(genesis.getTransactions().get(0));
21 }
```

```
1 public static void printBlockInfo(Block blk) throws InterruptedException {
2     System.out.println("Hash " + blk.getHashAsString());
3     System.out.println("Prev Hash " + blk.getPrevBlockHash());
4     System.out.println("Timestamp " + blk.getTimeSeconds());
5     System.out.println("Timestamp " + blk.time());
6 }
```

bitcoinj - transactions

Clear description of a transaction fields:

<https://dev.to/thunderbiscuit/deconstructing-a-bitcoin-transaction-4l2n>

Segwit recap

<https://learnmeabitcoin.com/technical/upgrades/segregated-witness/>

Questions

